

Depth Anything 3 User Guide

In order to start working with the model you can download the full GitHub repository folder (<https://github.com/ByteDance-Seed/Depth-Anything-3>) using the *git clone* command:

```
!git clone https://github.com/ByteDance-Seed/Depth-Anything-3.git
```

All the needed libraries can be installed using the *requirements.txt* file included in the downloaded folder:

```
%cd ../../Depth-Anything-3
!pip install -r requirements.txt
```

To avoid further incompatibilities between the just installed libraries, the use of the 1.26.4 version of NumPy is recommended.

```
!pip install "numpy<2.0,>=1.26.4" --upgrade --force-reinstall
```

Intending to shorten all the imports we will be doing, we can modify the python *sys.path* variable like this:

```
import sys
sys.path.append('/content/Depth-Anything-3/src')
```

Now we can initialize a DA3 network with a specified preset configuration using the *DepthAnything3* class on the [src/depth_anything_3/api.py](#) file as follows:

```
from depth_anything_3.api import DepthAnything3
model = DepthAnything3.from_pretrained("depth-anything/DA3METRIC-LARGE")
```

The preset configurations are found on Hugging Face Hub (<https://huggingface.co/depth-anything>) and can be loaded using the *from_pretrained* method. The names that can be used and the different tasks they perform are the following, in this file, although, we will focus on the process of obtaining the depth estimation:

Model Zoo

We release three series of models, each tailored for specific use cases in visual geometry.

-  **DA3 Main Series** (*DA3-Giant* , *DA3-Large* , *DA3-Base* , *DA3-Small*) These are our flagship foundation models, trained with a unified depth-ray representation. By varying the input configuration, a single model can perform a wide range of tasks:
 -  **Monocular Depth Estimation:** Predicts a depth map from a single RGB image.
 -  **Multi-View Depth Estimation:** Generates consistent depth maps from multiple images for high-quality fusion.
 -  **Pose-Conditioned Depth Estimation:** Achieves superior depth consistency when camera poses are provided as input.
 -  **Camera Pose Estimation:** Estimates camera extrinsics and intrinsics from one or more images.
 -  **3D Gaussian Estimation:** Directly predicts 3D Gaussians, enabling high-fidelity novel view synthesis.

-  **DA3 Metric Series** (`DA3Metric-Large`) A specialized model fine-tuned for metric depth estimation in monocular settings, ideal for applications requiring real-world scale.
-  **DA3 Monocular Series** (`DA3Mono-Large`). A dedicated model for high-quality relative monocular depth estimation. Unlike disparity-based models (e.g., [Depth Anything 2](#)), it directly predicts depth, resulting in superior geometric accuracy.

 Leveraging these available models, we developed a **nested series** (`DA3Nested-Giant-Large`). This series combines a any-view giant model with a metric model to reconstruct visual geometry at a real-world metric scale.

 Model Name	 Params	 Rel. Depth	 Pose Est.	 Pose Cond.	 GS	 Met. Depth	 Sky Seg	 License
Nested								
DA3NESTED-GIANT-LARGE-1.1	1.40B	✓	✓	✓	✓	✓	✓	CC BY-NC 4.0
DA3NESTED-GIANT-LARGE	1.40B	✓	✓	✓	✓	✓	✓	CC BY-NC 4.0
Any-view Model								
DA3-GIANT-1.1	1.15B	✓	✓	✓	✓			CC BY-NC 4.0
DA3-GIANT	1.15B	✓	✓	✓	✓			CC BY-NC 4.0
DA3-LARGE-1.1	0.35B	✓	✓	✓				CC BY-NC 4.0
DA3-LARGE	0.35B	✓	✓	✓				CC BY-NC 4.0
DA3-BASE	0.12B	✓	✓	✓				Apache 2.0
DA3-SMALL	0.08B	✓	✓	✓				Apache 2.0
Monocular Metric Depth								
DA3METRIC-LARGE	0.35B	✓				✓	✓	Apache 2.0
Monocular Depth								
DA3MONO-LARGE	0.35B	✓					✓	Apache 2.0

The Nested series uses an Any-view model to estimate pose and depth, and a monocular metric depth estimator for scaling.

 Models with the `-1.1` suffix are retrained after fixing a training bug; prefer these refreshed checkpoints. The original `DA3NESTED-GIANT-LARGE`, `DA3-GIANT`, and `DA3-LARGE` remain available but are deprecated. You could expect much better performance for street scenes with the `-1.1` models.

The files that contain the parameters of each configuration preset can be consulted in the [src/depth_anything_3/configs](#) directory. In them, the values of the parameters used to create the backbone (DinoV2) and the head (DPT or DualDPT) are specified:

Depth-Anything-3 / src / depth_anything_3 / configs /

bingykang init6c58

Name	Last commit message
..	
da3-base.yaml	init
da3-giant.yaml	init
da3-large.yaml	init
da3-small.yaml	init
da3metric-large.yaml	init
da3mono-large.yaml	init
da3nested-giant-large.yaml	init

main

Depth-Anything-3 / src / depth_anything_3 / configs / da3metric-large.yaml

bingykang init

CodeBlame

```
1  __object__:
2  path: depth_anything_3.model.da3
3  name: DepthAnything3Net
4  args: as_params
5
6  net:
7  __object__:
8  path: depth_anything_3.model.dinov2.dinov2
9  name: DinoV2
10  args: as_params
11
12  name: vitl
13  out_layers: [4, 11, 17, 23]
14  alt_start: -1 # -1 means disable
15  qknorm_start: -1
16  rope_start: -1
17  cat_token: False
18
19  head:
20  __object__:
21  path: depth_anything_3.model.dpt
22  name: DPT
23  args: as_params
24
25  dim_in: 1024
26  output_dim: 1
27  features: 256
28  out_channels: [256, 512, 1024, 1024]
```

The implementation of the model's backbone can be found in the: [src/depth_anything_3/model/dinov2](#) directory. There are four implementations of different sizes, defined in the [vision_transformer.py](#) file with the names: vit_base, vit_giant2, vit_large and vit_small. The different implementations can be selected respectively with the values "vits", "vitb", "vitl" and "vitg" with the *name* variable in the *DinoV2* class, found in the [dinov2.py](#) file.

The implementation of all the heads used in the model can be found in the different files in the [src/depth_anything_3/model](#) directory. In our case, since we focus mainly on obtaining depth estimation for single images, the most relevant one is the *DPT* head, which can be found in the [dpt.py](#) file.

The *DepthAnything3* class in [apy.py](#) builds the net using the preset configuration through the *load_config* and *create_object* functions, found on [src/depth_anything_3/cfg.py](#), a file containing a bunch of utility functions used as intermediaries that facilitate the use of the *DepthAnything3Net* class found on the [src/depth_anything_3/model/da3.py](#) file, the one that actually contains the model implementation.

Depth-Anything-3 / src / depth_anything_3 / api.py

Code

Blame

446 lines (390 loc) · 17.6 KB

```
48 class DepthAnything3(nn.Module, PyTorchModelHubMixin):
74
75     def __init__(self, model_name: str = "da3-large", **kwargs):
76         """
77         Initialize DepthAnything3 with specified preset.
78
79         Args:
80         model_name: The name of the model preset to use.
81             Examples: 'da3-giant', 'da3-large', 'da3metric-large', 'da3nested-giant-large'.
82         **kwargs: Additional keyword arguments (currently unused).
83         """
84         super().__init__()
85         self.model_name = model_name
86
87         # Build the underlying network
88         self.config = load_config(MODEL_REGISTRY[self.model_name])
89         self.model = create_object(self.config)
90         self.model.eval()
91
92         # Initialize processors
93         self.input_processor = InputProcessor()
94         self.output_processor = OutputProcessor()
95
96         # Device management (set by user)
97         self.device = None
98
99     @torch.inference_mode()
```

Depth-Anything-3 / src / depth_anything_3 / model / da3.py

Code

Blame

442 lines (382 loc) · 17.2 KB

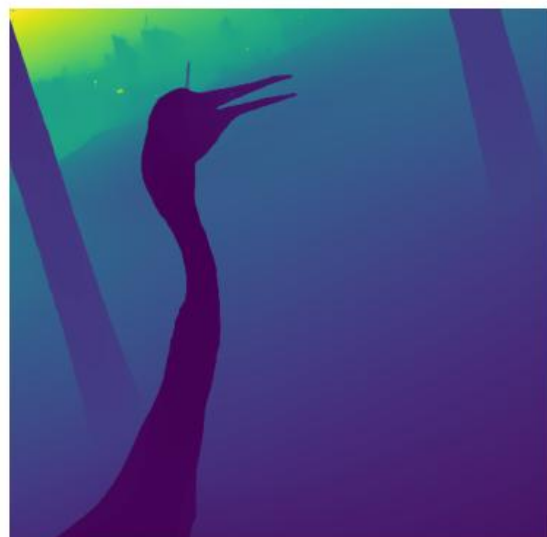
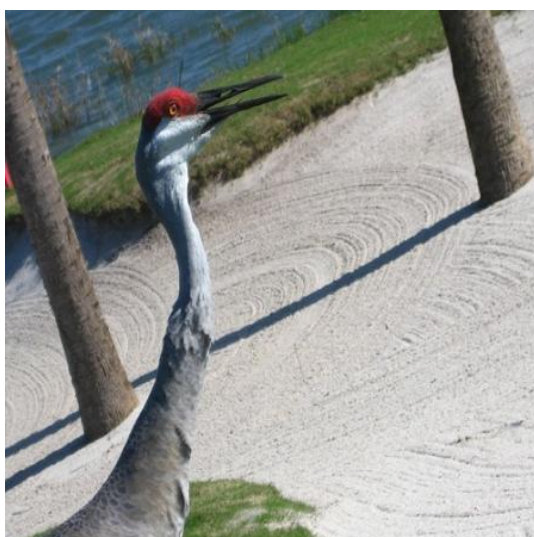
```
40 class DepthAnything3Net(nn.Module):
41     """
42     Depth Anything 3 network for depth estimation and camera pose estimation.
43
44     This network consists of:
45     - Backbone: DinoV2 feature extractor
46     - Head: DPT or DualDPT for depth prediction
47     - Optional camera decoders for pose estimation
48     - Optional GSDPT for 3DGS prediction
49
50     Args:
51         preset: Configuration preset containing network dimensions and settings
52
53     Returns:
54         Dictionary containing:
55         - depth: Predicted depth map (B, H, W)
56         - depth_conf: Depth confidence map (B, H, W)
57         - extrinsics: Camera extrinsics (B, N, 4, 4)
58         - intrinsics: Camera intrinsics (B, N, 3, 3)
59         - gaussians: 3D Gaussian Splats (world space), type: model.gs_adapter.Gaussians
60         - aux: Auxiliary features for specified layers
61     """
```

Once the *DA3* model has been initialized with one of the preset configurations using the *DepthAnything3* class from the [apy.py](#) file, the *inference* function can be used to pass an image through the entire model and obtain its depth estimation.

```
from depth_anything_3.api import DepthAnything3
model = DepthAnything3.from_pretrained("depth-anything/DA3METRIC-LARGE")
prediction_mono = model.inference(
    ["/content/00003.png"],
)

import matplotlib.pyplot as plt

plt.imshow(prediction_mono.depth.squeeze(0))
plt.axis('off')
plt.show()
```



To be able to work with the model and, for example, obtain the output after the backbone and pass it through two heads in parallel, one to obtain the depth estimation and the other to perform segmentation, it's necessary to analyse step by step everything that happens inside this *inference* function found in the [apy.py](#) file and be able to execute each step independently.

➤ Inference Function

Depth-Anything-3 / src / depth_anything_3 / api.py

Code

Blame

446 lines (390 loc) · 17.6 KB

```
48     class DepthAnything3(nn.Module, PyTorchModelHubMixin):
100
133     def inference(
134         self,
135         image: list[np.ndarray | Image.Image | str],
136         extrinsics: np.ndarray | None = None,
137         intrinsics: np.ndarray | None = None,
138         align_to_input_ext_scale: bool = True,
139         infer_gs: bool = False,
140         use_ray_pose: bool = False,
141         ref_view_strategy: str = "saddle_balanced",
142         render_exts: np.ndarray | None = None,
143         render_ixts: np.ndarray | None = None,
144         render_hw: tuple[int, int] | None = None,
145         process_res: int = 504,
146         process_res_method: str = "upper_bound_resize",
147         export_dir: str | None = None,
148         export_format: str = "mini_npz",
149         export_feat_layers: Sequence[int] | None = None,
150         # GLB export parameters
151         conf_thresh_percentile: float = 40.0,
152         num_max_points: int = 1_000_000,
153         show_cameras: bool = True,
154         # Feat_vis export parameters
155         feat_vis_fps: int = 15,
156         # Other export parameters, e.g., gs_ply, gs_video
157         export_kwargs: Optional[dict] = {},
158     ) -> Prediction:
```

```

162     Args:
163         image: List of input images (numpy arrays, PIL Images, or file paths)
164         extrinsics: Camera extrinsics (N, 4, 4)
165         intrinsics: Camera intrinsics (N, 3, 3)
166         align_to_input_ext_scale: whether to align the input pose scale to the prediction
167         infer_gs: Enable the 3D Gaussian branch (needed for `gs_ply`/`gs_video` exports)
168         use_ray_pose: Use ray-based pose estimation instead of camera decoder (default: False)
169         ref_view_strategy: Strategy for selecting reference view from multiple views.
170             Options: "first", "middle", "saddle_balanced", "saddle_sim_range".
171             Default: "saddle_balanced". For single view input ( $S \leq 2$ ), no reordering is performed.
172         render_exts: Optional render extrinsics for Gaussian video export
173         render_ixts: Optional render intrinsics for Gaussian video export
174         render_hw: Optional render resolution for Gaussian video export
175         process_res: Processing resolution
176         process_res_method: Resize method for processing
177         export_dir: Directory to export results
178         export_format: Export format (mini_npz, npz, glb, ply, gs, gs_video)
179         export_feat_layers: Layer indices to export intermediate features from
180         conf_thresh_percentile: [GLB] Lower percentile for adaptive confidence threshold (default: 40.0) # noqa: E501
181         num_max_points: [GLB] Maximum number of points in the point cloud (default: 1,000,000)
182         show_cameras: [GLB] Show camera wireframes in the exported scene (default: True)
183         feat_vis_fps: [FEAT_VIS] Frame rate for output video (default: 15)
184         export_kwargs: additional arguments to export functions.
185
186     Returns:
187         Prediction object containing depth maps and camera parameters
188     """

```

The structure of the *inference* function can be mainly divided into three parts: image preprocessing, forward pass through the model backbone and head, and postprocessing.

In the preprocessing part, it starts by calling the *_preprocess_inputs* function, which uses the *InputProcessor* class from the [src/depth_anything_3/utils/io/input_processor.py](#) file to convert the image or list of images into a tensor ready to be passed through the model, applying normalization and the necessary resizing.

Depth-Anything-3 / src / depth_anything_3 / utils / io / input_processor.py

Code Blame 501 lines (445 loc) · 18.5 KB

```

35     class InputProcessor:
41         Pipeline:
42             1) Load image and convert to RGB
43             2) Boundary resize (upper/lower bound, preserving aspect ratio)
44             3) Enforce divisibility by PATCH_SIZE:
45                 - "*resize" methods: each dimension is rounded to nearest multiple
46                   (may up/downscale a few px)
47                 - "*crop" methods: each dimension is floored to nearest multiple via center crop
48             4) Convert to tensor and apply ImageNet normalization
49             5) Stack into (1, N, 3, H, W)

```

Next, it uses the *_prepare_model_inputs* function to move the previously generated tensor to the same device as the model, and it also converts the camera extrinsics and intrinsics into tensors (in case they are used). Finally, the preprocessing stage calls the *_normalize_extrinsics* function to normalize the tensor containing the extrinsics.

```

imgs_cpu, extrinsics, intrinsics = model._preprocess_inputs(["/content/00003.png"])
imgs, ex_t, in_t = model._prepare_model_inputs(imgs_cpu, extrinsics, intrinsics)
ex_t_norm = model._normalize_extrinsics(ex_t)

```

Once the preprocessing stage is finished, the *inference* function performs a forward pass through the backbone and the head using the *DepthAnything3Net* class in the [da3.py](#) file mentioned earlier. To obtain the backbone and the head of the model separately, there are two options. If you want to use the same parameter values for both the backbone and the head as in one of the preset configurations discussed at the beginning of the file, the full model can be loaded, and the backbone and head can simply be selected as follows:

```

from depth_anything_3.api import DepthAnything3

model = DepthAnything3.from_pretrained("depth-anything/da3metric-large")

net = model.model

backbone_ = net.backbone
head_ = net.head

for p in backbone_.parameters():
    p.requires_grad = False

for p in head_.parameters():
    p.requires_grad = False

backbone_.eval()
head_.eval()

features,_ = backbone_(imgs)
out = head_(features, imgs.shape[-2], imgs.shape[-1], 0)

```

In the previous code, it is important to note that the backbone returns a tuple, but the head only needs to receive the first element of this tuple.

If, instead, you want to modify the model parameters, you need to directly create the backbone and the head using the [dinov2.py](#) and the [dpt.py](#) files that contain their respective implementations. In this case, if you want to work with the weights obtained after training *DA3*, where fine-tuning was performed on top of the *DinoV2* weights, it's important that you load the weights of the model trained with the preset configuration. The following code shows how to do this easily for the backbone, for the head, it would be completely equivalent using the corresponding [dpt.py](#).

```

from depth_anything_3.model.dinov2.dinov2 import DinoV2

backbone = DinoV2(
    name = "vitl",
    out_layers= [4, 11, 17, 23],
    alt_start= -1,
    qknorm_start= -1,
    rope_start= -1,
    cat_token= False,
)

for p in backbone.parameters():
    p.requires_grad = False

from depth_anything_3.api import DepthAnything3

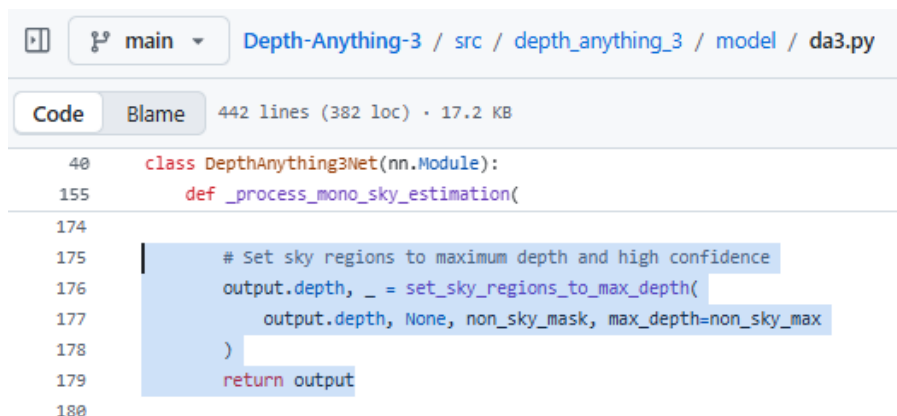
model = DepthAnything3.from_pretrained("depth-anything/da3metric-large")

net = model.model

backbone.load_state_dict(backbone_.state_dict())

```

Although we have said that in our case we will focus on the process needed to obtain the depth approximation, if we analyse the [da3.py](#) file where the model's forward pass is defined, we can see that during this process the `_process_mono_sky_estimation` function is called. This function is used to classify which pixels correspond to sky regions and which do not. However, within this function, the depth values of pixels classified as sky are slightly adjusted. Therefore, the output of our head must also be passed through it in order to obtain the same depth approximation as when using the inference function.



The screenshot shows a code editor with the file path `Depth-Anything-3 / src / depth_anything_3 / model / da3.py`. The editor displays the `DepthAnything3Net` class and the `_process_mono_sky_estimation` function. The function is defined at line 155 and includes a comment at line 175: `# Set sky regions to maximum depth and high confidence`. The function calls `set_sky_regions_to_max_depth` at line 176, passing `output.depth`, `None`, `non_sky_mask`, and `max_depth=non_sky_max`. The function returns `output` at line 179.

```

40     class DepthAnything3Net(nn.Module):
155         def _process_mono_sky_estimation(
174
175             # Set sky regions to maximum depth and high confidence
176             output.depth, _ = set_sky_regions_to_max_depth(
177                 output.depth, None, non_sky_mask, max_depth=non_sky_max
178             )
179             return output
180

```

```

features,_ = backbone_(imgs)
out = head_(features,imgs.shape[-2], imgs.shape[-1],0)
output = net._process_mono_sky_estimation(out)

```

In the postprocessing stage, the `_convert_to_prediction` function is called, which uses the `OutputProcessor` class from the [src/depth anything 3/utils/io/output_processor.py](#) file to convert the output tensor to NumPy, remove the batch dimension, and organize the different elements of the output into an object belonging to the `Prediction` class defined in the [src/depth anything 3/specs.py](#) file.

```
35  class Prediction:
36      depth: np.ndarray # N, H, W
37      is_metric: int
38      sky: np.ndarray | None = None # N, H, W
39      conf: np.ndarray | None = None # N, H, W
40      extrinsics: np.ndarray | None = None # N, 4, 4
41      intrinsics: np.ndarray | None = None # N, 3, 3
42      processed_images: np.ndarray | None = None # N, H, W, 3 - processed images for visualization
43      gaussians: Gaussians | None = None # 3D gaussians
44      aux: dict[str, Any] = None #
45      scale_factor: Optional[float] = None # metric scale
```

Finally, the inference function calls the `_align_to_input_extrinsics_intrinsics` function, which forces the predicted camera geometry into the coordinate system of the reference extrinsics using similarity alignment, optionally correcting the depth scale, and the `_add_processed_images` function, which adds processed images to the prediction for visualization, reordering their dimensions and denormalizing them.

```
prediction = model._convert_to_prediction(out)
prediction = model._align_to_input_extrinsics_intrinsics(extrinsics, intrinsics, prediction)
prediction = model._add_processed_images(prediction, imgs_cpu)
```